

**UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO – UFRPE BACHARELADO
EM SISTEMAS DE INFORMAÇÃO**

MARCOS GUILHERME OLIVEIRA LIMA

**GESTÃO ATÍPICOS: SISTEMA WEB PARA GESTÃO ESPECIALIZADA DE ALUNOS
ATÍPICOS NA EDUCAÇÃO INCLUSIVA**

Belo Jardim – PE

2026

MARCOS GUILHERME OLIVEIRA LIMA

**GESTÃO ATÍPICOS: SISTEMA WEB PARA GESTÃO ESPECIALIZADA DE ALUNOS
ATÍPICOS NA EDUCAÇÃO INCLUSIVA**

*Trabalho de Conclusão de Curso apresentado ao
Bacharelado em Sistemas de Informação da
Universidade Federal Rural de Pernambuco, como
requisito parcial para obtenção do título de
Bacharel em Sistemas de Informação.*

Orientador: A definir

Belo Jardim - PE

2026

Sumário

RESUMO	4
ABSTRACT.....	5
1. INTRODUÇÃO.....	6
2. REFERENCIAL TEÓRICO.....	7
3. METODOLOGIA.....	10
4. ESCOPO DOS TESTES DE PENETRAÇÃO.....	15
5. EXECUÇÃO E ANÁLISE DOS TESTES DE SEGURANÇA.....	18
6. CONCLUSÃO	20
REFERÊNCIAS	21
Anexo A – Capturas de Tela	22
Anexo B – Relatório de Pentest.....	24

Resumo

O presente trabalho apresenta o desenvolvimento de um sistema web denominado Gestão Atípicos, destinado à gestão especializada de alunos atípicos no contexto da educação inclusiva. A aplicação permite o cadastramento, acompanhamento individualizado, upload de laudos médicos, registro de observações diárias, agenda de cuidados, gestão do Plano de Desenvolvimento Individual (PDI), controle rigoroso de medicações (Diário de Saúde) e geração de relatórios, com perfis diferenciados. O sistema prioriza a segurança da informação por meio de Row Level Security (RLS), Autenticação em Duas Etapas (2FA/MFA), conformidade com a LGPD, testes de penetração e modelo de ameaças STRIDE. O objetivo foi criar uma solução prática para escolas que ainda utilizam planilhas e documentos físicos, promovendo maior eficiência e inclusão. O sistema encontra-se implantado e disponível publicamente para testes.

Palavras-chave: educação inclusiva, alunos atípicos, sistema de gestão, segurança da informação, LGPD, Supabase.

Abstract

This paper presents the development of a web-based system called **Gestão Atípicos**, designed for the specialized management of atypical students within the context of inclusive education. The application enables student registration, individualized monitoring, medical report uploads, daily observation records, care scheduling, Individual Development Plan (IDP) management, strict medication control through a Health Diary, and report generation, with differentiated user profiles. The system prioritizes information security through Row Level Security (RLS), Two-Factor Authentication (2FA/MFA), compliance with the Brazilian General Data Protection Law (LGPD), penetration testing, and the STRIDE threat modeling framework. The objective was to create a practical solution for schools that still rely on spreadsheets and physical documents, promoting greater efficiency, organization, and inclusion. The system has been deployed and is publicly available for testing.

Keywords: inclusive education, atypical students, management system, information security, LGPD, Supabase.

1. INTRODUÇÃO

1.1 Contextualização:

A educação inclusiva no Brasil é obrigatória por lei (Lei nº 13.146/2015 - Estatuto da Pessoa com Deficiência e Política Nacional de Educação Especial). No entanto, muitas instituições ainda gerenciam alunos com necessidades educacionais especiais (atípicos) de forma manual, utilizando planilhas eletrônicas e documentos físicos. Essa prática gera perda de informação, dificuldade de comunicação entre equipe e família, e risco de não conformidade com a LGPD (Lei nº 13.709/2018).

1.2 Problema de Pesquisa:

Como desenvolver um sistema web seguro, acessível e funcional que centralize a gestão de alunos atípicos, facilitando o trabalho de gestores, cuidadores e responsáveis?

1.3 Objetivos Objetivo Geral:

Desenvolver e implantar um sistema web para gestão especializada de alunos atípicos.

Objetivos Específicos:

Implementar perfis de acesso diferenciados;

Garantir conformidade com LGPD e boas práticas de segurança;

Permitir importação/exportação em massa via planilhas;

Disponibilizar o sistema em produção para testes reais.

1.4 Justificativa:

O projeto possui relevância social, ao promover a inclusão escolar através da tecnologia, e relevância técnica, pela aplicação prática de conceitos de engenharia de software, segurança da informação e desenvolvimento *full-stack*.

2. REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento do sistema "Gestão Atípicos", abrangendo desde as bases legais da educação inclusiva e proteção de dados até as tecnologias e padrões de segurança de software utilizados.

2.1 Educação Inclusiva e Atendimento Educacional Especializado (AEE):

A educação inclusiva é um paradigma que visa garantir o acesso, a permanência e a aprendizagem de todos os alunos, independentemente de suas condições físicas, intelectuais ou sensoriais. No Brasil, a Política Nacional de Educação Especial (PNEE) e o Estatuto da Pessoa com Deficiência (Lei nº 13.146/2015) estabelecem que o sistema educacional deve ser inclusivo em todos os níveis. Nesse contexto, o Atendimento Educacional Especializado (AEE) surge não como um substituto ao ensino regular, mas como um serviço complementar ou suplementar. O AEE identifica, elabora e organiza recursos pedagógicos e de acessibilidade que eliminem as barreiras para a plena participação dos alunos, considerando suas necessidades específicas (BRASIL, 2020). A gestão eficiente desses dados é crucial para garantir que o plano de ensino individualizado seja cumprido.

2.2 Legislação e Privacidade de Dados (LGPD):

A Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018) impõe rigorosas diretrizes sobre a coleta e tratamento de informações pessoais. Quando se trata de dados de crianças e adolescentes, o Artigo 14 da lei exige que o tratamento seja realizado em seu "melhor interesse", mediante consentimento específico de um dos pais ou responsável legal. No ambiente escolar, dados de saúde (laudos médicos, terapias, diagnósticos) são classificados como dados sensíveis. O armazenamento dessas informações em meios físicos ou planilhas sem controle de acesso adequado expõe as instituições a riscos elevados de vazamento e incidentes de segurança, tornando mandatória a adoção de sistemas que garantam a confidencialidade e a integridade desses registros.

2.3 Segurança de Aplicações Web e Padrões OWASP:

A segurança no desenvolvimento de software é orientada por padrões internacionais. A *Open Web Application Security Project* (OWASP) é uma fundação global que documenta os riscos mais críticos para aplicações web. O documento OWASP Top 10 lista as vulnerabilidades mais comuns, como Injeção de Código, Falhas de Autenticação e Configuração Insegura. Para garantir um nível de segurança verificável, utiliza-se o ASVS (*Application Security Verification Standard*). O ASVS fornece uma base para testar controles de segurança técnica em aplicações web, oferecendo uma lista de requisitos que desenvolvedores e auditores podem usar para assegurar que a aplicação é segura por *design*, mitigando vetores de ataque conhecidos.

2.4 Modelagem de Ameaças (STRIDE):

A modelagem de ameaças é uma abordagem proativa para identificar falhas de segurança ainda na fase de design do software. A metodologia STRIDE, desenvolvida pela Microsoft, é utilizada para categorizar ameaças em seis tipos:

- Spoofing (Falsificação de identidade);
- Tampering (Violação de dados);
- Repudiation (Repúdio);
- Information Disclosure (Divulgação de informações);
- Denial of Service (Negação de serviço);
- Elevation of Privilege (Elevação de privilégio). A aplicação deste modelo permite mapear como um atacante poderia explorar o sistema, orientando a implementação de contramedidas específicas antes mesmo da escrita do código.

2.5 Controle de Acesso e Row Level Security (RLS):

O controle de acesso é o mecanismo que limita as ações de um usuário autenticado dentro do sistema. Em arquiteturas modernas de banco de dados, destaca-se o uso de Row Level Security (RLS) ou Segurança em Nível de Linha. Diferente da validação tradicional feita apenas no *backend* (servidor), o RLS aplica as regras de autorização diretamente no banco de dados. Isso garante que, mesmo que a aplicação sofra uma falha na camada de API, o banco de dados impedirá que um usuário acesse registros (linhas) que não pertencem ao seu perfil ou organização (tenant). Essa técnica é fundamental para sistemas multi-inquilinos (SaaS) que lidam com dados sensíveis.

2.6 Arquitetura Moderna e Serverless:

Para atender aos requisitos de escalabilidade e manutenibilidade, optou-se por uma arquitetura baseada em microsserviços e computação em nuvem. O desenvolvimento *frontend* utiliza a biblioteca React, permitindo a criação de Interfaces de Usuário (UI) reativas e baseadas em componentes. No *backend*, adota-se o modelo Serverless (BaaS - *Backend as a Service*) através da plataforma Supabase. Essa abordagem remove a necessidade de gerenciamento manual de servidores, delegando a infraestrutura, a autenticação e o banco de dados para serviços gerenciados, o que reduz custos operacionais e permite foco total nas regras de negócio e na segurança da aplicação.

2.7 Autenticação Multifator (MFA/2FA):

A Autenticação Multifator é um mecanismo de segurança que exige duas ou mais provas de identidade (fatores) antes de conceder acesso a um sistema. Segundo as diretrizes da OWASP, depender exclusivamente de senhas (fator de conhecimento) é insuficiente contra ataques modernos como credential stuffing ou phishing. A implementação do MFA, utilizando aplicativos geradores de códigos baseados em tempo (TOTP - Time-Based One-Time Password), como o Google Authenticator, adiciona um fator de posse (o dispositivo móvel do usuário), elevando drasticamente a resiliência do sistema contra acessos não autorizados, especialmente em plataformas que lidam com dados sensíveis de saúde e educação infantil

3. METODOLOGIA

A presente pesquisa classifica-se como aplicada, com abordagem qualitativa, voltada para o desenvolvimento de uma solução tecnológica. Para a condução do projeto, adotou-se o desenvolvimento ágil de software, caracterizado por entregas incrementais e iterativas. O processo foi estruturado em ciclos semanais (*sprints*), permitindo feedback contínuo e ajustes rápidos de escopo. As etapas metodológicas foram divididas conforme descrito a seguir.

3.1 Levantamento de Requisitos e Elicitação:

A etapa inicial consistiu na elicitação de requisitos funcionais e não funcionais. Para tanto, foram realizadas entrevistas não estruturadas (conversas abertas) com profissionais da área da educação. O objetivo dessas interações foi mapear as reais necessidades do público-alvo, identificar "dores" nos processos atuais e validar a viabilidade da proposta. As informações coletadas foram documentadas e priorizadas para orientar o desenvolvimento.

3.2 Design de Interface e Modelagem de Dados:

Posteriormente, iniciou-se a fase de projeto. No âmbito da interface (UI/UX), buscou-se criar protótipos focados na usabilidade e na experiência do usuário. Paralelamente, foi realizada a modelagem do banco de dados, definindo-se as entidades e relacionamentos necessários para garantir a integridade e a persistência das informações do sistema.

3.3 Desenvolvimento e Implantação Contínua (CI/CD):

A codificação seguiu a abordagem iterativa. Para garantir a agilidade e a qualidade do código, utilizou-se um fluxo de Integração e Entrega Contínuas (Continuous Integration/Continuous Deployment - CI/CD).

3.3.1 Controle de Versão: O código foi gerenciado via Git/GitHub.

3.3.2 Automação: Utilizou-se o GitHub Actions para automatizar rotinas de build e testes a cada novo commit.

3.3.3 Hospedagem: A implantação (deploy) foi realizada na plataforma Vercel, permitindo a disponibilização imediata das novas funcionalidades em ambiente de produção.

3.4 Estratégia de Testes e Segurança Para assegurar a robustez da aplicação, foram aplicadas diferentes camadas de verificação:

3.4.1 Testes Unitários: Para validar a lógica de componentes isolados do sistema.

3.4.2 Testes de Integração: Para garantir que os módulos do sistema funcionem corretamente quando combinados.

3.4.3 Resultados dos Testes Automatizados:

Os testes automatizados foram implementados utilizando o framework Vitest, que permitiu verificar o comportamento dos componentes e funcionalidades críticas da aplicação. Os testes foram organizados em dois arquivos principais:

src/___tests___/auth.test.tsx: Responsável por testar o fluxo de autenticação, incluindo o login, exibição de mensagens de erro e tratamento de erros inesperados.

src/___tests___/NewNoteDialog.test.tsx: Responsável por testar o componente de diálogo para criação de novas observações, incluindo a validação de campos obrigatórios e o estado de carregamento durante a submissão.

Resultados Detalhados

A execução dos testes resultou em 14 testes aprovados, distribuídos da seguinte forma:

src/___tests___/auth.test.tsx: 9 testes aprovados

- deve corresponder ao snapshot da UI

- deve permitir que o usuário digite credenciais e clique em entrar

- deve exibir mensagem de erro quando as credenciais são inválidas

- deve tratar erros inesperados no login (catch block)

- deve redirecionar (não renderizar o form) se o usuário já estiver logado

- deve baixar o APK ao clicar em instalar no Android

- deve mostrar instrução de instalação no iOS

- deve usar o prompt de instalação PWA se disponível (Desktop/Mobile)

- deve baixar o APK como fallback em Desktop/Outros (sem PWA prompt)

src/___tests___/NewNoteDialog.test.tsx: 5 testes aprovados

- deve renderizar corretamente quando aberto

- deve validar campos obrigatórios (impedir envio vazio)

- deve validar o comprimento mínimo da nota

- deve chamar a mutação com os dados corretos ao submeter formulário válido

deve exibir estado de carregamento durante a submissão

Observações

Durante a execução dos testes, foram identificados alguns avisos relacionados ao React Router, indicando a necessidade de avaliar a compatibilidade com futuras versões da biblioteca.

3.5 Desenvolvimento da Aplicação e Mobilidade (PWA)

O "Gestão Atípicos", foi concebido sob o paradigma de **Progressive Web App (PWA)**. Esta abordagem arquitetural foi selecionada para garantir que o sistema possua a onipresença da web aliada à experiência de uso de aplicativos nativos, atendendo à necessidade de acessibilidade multiplataforma (desktop e mobile) sem a complexidade de manutenção de códigos-fonte distintos para cada sistema operacional.

A construção da aplicação utilizou a biblioteca **React** em conjunto com a linguagem **TypeScript**, proporcionando uma interface de usuário reativa e um código mais seguro e escalável devido à tipagem estática. Para a compilação e otimização dos recursos, adotou-se a ferramenta **Vite**.

A característica de "aplicativo" foi implementada através de duas estratégias principais:

1. **Service Workers e Manifesto Web:** Utilizando o plugin vite-plugin-pwa, foi configurado um *Service Worker* para atuar como proxy de rede no navegador do cliente. Isso permite o cacheamento inteligente de ativos estáticos (HTML, CSS, JavaScript) e requisições, garantindo que a aplicação carregue instantaneamente e ofereça suporte a funcionamento offline ou em redes instáveis, cenário comum em ambientes escolares. O *Web App Manifest* foi configurado para definir a identidade visual do aplicativo (ícones, cores de tema) quando instalado na tela inicial do usuário.
2. **Empacotamento Nativo (Capacitor):** Para a distribuição em dispositivos móveis Android, integrou-se o **Capacitor** ao fluxo de desenvolvimento. O Capacitor atua como uma ponte (*bridge*) que encapsula a aplicação web compilada em um contêiner nativo. Isso permitiu a geração de um arquivo instalável (**APK**), possibilitando que o sistema "Gestão Atípicos" seja executado como um aplicativo Android nativo, com acesso potencial a recursos do dispositivo e desempenho otimizado, mantendo a mesma base de código da versão web.

3.6 Desenvolvimento de Aplicação Nativa (Android APK)

O desenvolvimento da versão móvel nativa do sistema **Gestão Atípicos** foi realizado através da utilização do **Capacitor**, um *runtime* multiplataforma que permite a execução de aplicações web

modernas em dispositivos nativos. Esta abordagem estratégica possibilitou o reaproveitamento integral da base de código desenvolvida em **React** e **TypeScript**, garantindo a consistência visual e funcional entre as versões web e mobile, além de reduzir significativamente o tempo e o custo de desenvolvimento.

A arquitetura da solução móvel consiste no encapsulamento da aplicação web (SPA - *Single Page Application*) dentro de uma *WebView* nativa do sistema operacional Android. O Capacitor atua como uma ponte (*bridge*), permitindo que o código JavaScript se comunique com APIs nativas do dispositivo quando necessário, mantendo o desempenho próximo ao de aplicações nativas puras para operações de interface.

O processo de geração do artefato instalável (APK - *Android Package Kit*) foi estruturado em um *pipeline* de construção automatizado, compreendendo as seguintes etapas:

1. **Compilação Web:** A aplicação é transpilada e otimizada pela ferramenta de build **Vite**, gerando os arquivos estáticos (HTML, CSS, JavaScript) minificados no diretório de distribuição.
2. **Sincronização Nativa:** O comando `npx cap sync` sincroniza os arquivos estáticos gerados para o diretório de *assets* do projeto Android (`android/app/src/main/assets/public/`) e atualiza as dependências de plugins nativos.
3. **Geração de Recursos Gráficos:** Para assegurar uma experiência de usuário consistente com as diretrizes de design do Android, ícones adaptativos e telas de abertura (*Splash Screens*) são gerados automaticamente para diversas densidades de tela (de *ldpi* a *xxxhdpi*) utilizando a ferramenta `@capacitor/assets`.
4. **Compilação Nativa:** O sistema de build **Gradle** é utilizado para compilar o projeto Android, empacotando o *runtime* do Capacitor, os plugins e os *assets* web em um arquivo `.apk` assinado, pronto para instalação.

Adicionalmente, para facilitar a distribuição e o acesso em ambientes onde a publicação em lojas de aplicativos (Google Play Store) pode ser burocrática ou desnecessária (como em testes internos ou uso corporativo restrito), foi implementado um mecanismo de **distribuição direta**. A aplicação web detecta, através do *User Agent*, se o acesso está sendo realizado por um dispositivo Android. Caso positivo, e se a aplicação não estiver rodando em modo *standalone* (instalada), a interface oferece proativamente um link direto para o download do arquivo APK hospedado no servidor, permitindo a instalação imediata pelo usuário (*Sideload*).

Esta estratégia híbrida conferiu ao projeto **Gestão Atípicos** a versatilidade de uma aplicação web com a presença e a capacidade de integração de uma aplicação nativa, atendendo aos requisitos de acessibilidade e mobilidade dos cuidadores e responsáveis.

3.6 Módulos Integrados do Sistema

Para abranger a jornada completa do estudante atípico e atender aos requisitos levantados, a plataforma foi estruturada em módulos altamente integrados:

PDI e Evolução: Painel analítico que gamifica o progresso do aluno (sistema de conquistas), mapeia metas semestrais e cruza gatilhos comportamentais, consolidando o anedotário multidisciplinar.

Diário de Saúde: Módulo focado no controle rigoroso da rotina fisiológica e administração de medicamentos, permitindo que a escola controle horários e dosagens de forma segura.
Comunicação Segura: Portal centralizado que substitui aplicativos de mensagens não oficiais, oferecendo chat criptografado, mural de recados com controle de ciência e gestão de reuniões virtuais.

Autenticação Avançada (2FA): Exigência de vínculo com aplicativos autenticadores (ex: Google Authenticator) durante o login, garantindo que mesmo em caso de vazamento de senhas, os laudos e dados médicos dos alunos permaneçam protegidos pelo segundo fator

4. ESCOPO DOS TESTES DE PENETRAÇÃO

Após a execução dos ciclos de testes de segurança, baseados na metodologia descrita anteriormente e nas diretrizes do OWASP, os resultados foram compilados e analisados. O objetivo desta seção é discutir a eficácia dos controles de segurança implementados na plataforma "Gestão Atípicos", avaliando a robustez da arquitetura baseada em Vercel e Supabase.

A auditoria, finalizada em dezembro de 2026, indicou que a aplicação atingiu um nível satisfatório de maturidade de segurança, não apresentando vulnerabilidades críticas exploráveis no escopo avaliado impedindo que um atacante deduza quais usuários estão cadastrados na base de dados.

4.1 Análise de Autenticação e Gestão de Sessão

Os testes focados no mecanismo de autenticação (Supabase Auth) demonstraram resiliência contra ataques comuns de força bruta e enumeração. Observou-se que a aplicação responde com mensagens genéricas de erro ao processar credenciais inválidas ou e-mails inexistentes, impedindo que um atacante deduza quais usuários estão cadastrados na base de dados.

Destaca-se a implementação mandatória da Autenticação em Duas Etapas (2FA) via TOTP. Durante os testes de segurança, simulou-se o comprometimento e vazamento da senha legítima de um usuário 'Gestor'. Mesmo possuindo a credencial correta de primeiro fator, o acesso do atacante foi integralmente neutralizado pela ausência do token dinâmico, comprovando a resiliência do sistema contra ataques de roubo e reaproveitamento de senhas.

No que tange à gestão de sessões, a validação de tokens JWT (*JSON Web Tokens*) mostrou-se eficaz. Tentativas de injeção de tokens incompletos, não assinados ou com assinaturas inválidas resultaram invariavelmente em respostas de acesso negado (401 Unauthorized). Além disso, o fluxo de recuperação de senha (/recover) foi validado, garantindo que não há exposição de informações sensíveis durante o processo de redefinição de credenciais.

4.2 Validação de Controle de Acesso e Row Level Security (RLS)

Um ponto crítico da arquitetura do Supabase é a exposição intencional da chave pública (anon key) no frontend. A análise de segurança confirmou que essa característica não configura uma vulnerabilidade, visto que a segurança da aplicação não depende do sigilo desta chave, mas sim das políticas de segurança em nível de linha (Row Level Security - RLS).

Os testes de acesso direto à API REST (/rest/v1/) e as tentativas de escalação de privilégios corroboraram a eficácia do RLS:

Isolamento de Dados: Tentativas de leitura ou escrita em tabelas sem o token de autenticação adequado foram bloqueadas.

Escalação Vertical: Testes simulando um usuário com perfil de "Cuidador" tentando elevar seus privilégios para "Gestor" (via manipulação do parâmetro role em requisições PATCH) falharam. O sistema ignorou silenciosamente os campos injetados ou bloqueou a operação, retornando que zero linhas foram afetadas.

Isso evidencia que a lógica de autorização está corretamente dissociada do cliente e rigidamente aplicada na camada de banco de dados.

4.3 Segurança da Camada de Aplicação e Infraestrutura

A avaliação do frontend e das configurações HTTP indicou a conformidade com práticas modernas de defesa. A injeção de payloads maliciosos visando Cross-Site Scripting (XSS) — incluindo vetores em SVG e query strings — não obteve sucesso, demonstrando que a sanitização de inputs e a renderização segura dos componentes (Vercel/React) estão operantes.

Adicionalmente, a análise dos cabeçalhos HTTP revelou a presença de configurações de endurecimento (*hardening*), como Strict-Transport-Security (HSTS) e X-Content-Type-Options: nosniff. A infraestrutura, suportada pela Vercel e Cloudflare, provou-se resiliente contra tentativas básicas de negação de serviço, delegando o *rate limiting* para a borda (*edge*).

4.4 Estratégias de Defesa Ativa e Monitoramento

Diferenciando-se de implementações padrão, o projeto adotou uma postura de "Defesa em Profundidade" (Defense in Depth) através da implementação de mecanismos de defesa ativa.

Honeytokens e Canary Endpoints: Foram inseridos artefatos falsos (credenciais e rotas de API) para atuar como armadilhas. Qualquer interação com esses elementos aciona alertas de segurança, permitindo a detecção precoce de varreduras ou vazamentos.

Rastreabilidade: A implementação de *Correlation IDs* (Identificadores de Correlação) permitiu o rastreamento granular de requisições. Em caso de anomalia, é possível isolar a transação exata nos logs, facilitando a resposta a incidentes.

Monitoramento: A integração com um SIEM (*Security Information and Event Management*) dedicado assegura que eventos críticos sejam correlacionados e monitorados em tempo real.

4.5 Síntese dos Resultados

O quadro a seguir resume os vetores de ataque testados e o status de vulnerabilidade encontrado.

Quadro 1 – Resumo da Auditoria de Segurança

Vetor de Ataque	Status	Observação
SQL Injection	Mitigado	Uso de ORM/API segura e RLS.
XSS (<i>Cross-Site Scripting</i>)	Mitigado	Sanitização automática do framework.
Quebra de Autenticação	Mitigado	Supabase Auth com validação robusta.
Exposição de Dados Sensíveis	Mitigado	RLS impede vazamento de dados de terceiros.
Escalação de Privilégios	Mitigado	Validação de <i>roles</i> no banco de dados.
Configuração de Segurança Incorreta	Mitigado	Headers de segurança e HTTPS forçado ativos.

Fonte: Elaborado pelo autor (2026).

Conclui-se que a arquitetura de segurança proposta atende aos requisitos de confidencialidade, integridade e disponibilidade, tendo sido aprovada em todos os testes do *checklist* OWASP ASVS aplicáveis ao escopo do projeto.

5. EXECUÇÃO E ANÁLISE DOS TESTES DE SEGURANÇA

Esta seção detalha os procedimentos técnicos executados para validar a segurança da aplicação "Gestão Atípicos". Os testes foram conduzidos seguindo a metodologia de caixa preta (*Black-Box Testing*), simulando o comportamento de um agente mal-intencionado externo.

5.1 Autenticação e Autorização:

Foram realizados testes exaustivos nos mecanismos de login e gestão de identidade. As simulações incluíram tentativas de acesso com credenciais inexistentes, senhas incorretas e e-mails em formatos válidos e inválidos. Adicionalmente, executaram-se testes de enumeração de usuários (tentativa de descobrir quais e-mails estão cadastrados baseando-se no tempo de resposta ou mensagens de erro) e exploração direta da API de autenticação. Os resultados demonstraram a robustez do sistema:

Enumeração Mitigada: O sistema não retornou mensagens distintas para "usuário não encontrado" ou "senha incorreta", utilizando respostas genéricas que impedem a enumeração de contas.

Validação de Chaves: A autenticação provou-se protegida pela chave pública (anon key) em conjunto com tokens de sessão válidos, rejeitando requisições malformadas.

5.2 Análise de Exposição de Chaves:

(Supabase) Durante a análise estática do código *frontend*, identificou-se a presença da chave de API pública (anon key). Procedeu-se, então, à verificação de impacto dessa exposição. A análise técnica confirmou que a arquitetura do Supabase utiliza essa chave intencionalmente para iniciar a conexão, não conferindo, por si só, privilégios administrativos. Os testes práticos confirmaram que a segurança depende exclusivamente das políticas de *Row Level Security* (RLS). As tentativas de leitura e escrita utilizando apenas a anon key, sem um token de usuário autenticado, resultaram invariavelmente no código de resposta HTTP 401 Unauthorized, confirmando que não há vulnerabilidade de exposição indevida.

5.3 Integridade de Tokens JWT:

A integridade da gestão de sessões foi avaliada através da manipulação de *JSON Web Tokens* (JWT). Foram injetados tokens incompletos, reaproveitados de sessões expiradas, sem assinatura digital válida e com cabeçalhos Authorization forjados. Em todos os cenários, o *middleware* de segurança do Supabase rejeitou as requisições, impossibilitando qualquer tentativa de fixação de sessão ou escalada de privilégio via token.

5.4 Segurança da API REST:

Realizaram-se tentativas de acesso direto aos *endpoints* da API (/rest/v1/) contornando a interface gráfica. Os testes utilizaram chaves de API inválidas e requisições anônimas. O sistema manteve o comportamento seguro, negando o acesso (401 Unauthorized) e garantindo que nenhuma tabela ou *view* do banco de dados fosse exposta publicamente sem a devida autorização.

5.5 Testes de Escalação de Privilégios

Para validar o isolamento entre perfis de usuários, tentou-se a escalação vertical de privilégios (ex.: um perfil "Cuidador" tentando atuar como "Gestor"). Os testes envolveram:

Manipulação de Parâmetros: Alteração do campo `role` via requisição `PATCH /profiles`.

Mass Assignment: Tentativa de atribuição em massa em campos sensíveis.

As políticas de RLS bloquearam efetivamente as alterações. O servidor ignorou silenciosamente os campos injetados ou retornou que "0 linhas foram afetadas", mantendo o perfil do usuário inalterado e preservando a integridade dos dados.

5.6 Prevenção contra Cross-Site Scripting (XSS)

A resiliência da interface foi testada contra injeção de scripts maliciosos. Aplicaram-se *payloads* clássicos (como `<script>alert(1)</script>` e vetores SVG) em campos de entrada, *query strings* e no corpo JSON das requisições. Não foi observada execução de código arbitrário nem reflexão de entrada (*Reflected XSS*), indicando que as rotinas de sanitização do *framework* e da plataforma Vercel estão operantes.

5.7 Cabeçalhos de Segurança (Hardening)

A análise das respostas HTTP evidenciou a presença de cabeçalhos de segurança modernos, essenciais para a proteção do navegador do usuário. Foram confirmados:

Strict-Transport-Security (HSTS): Forçando a comunicação via HTTPS.

X-Content-Type-Options: `nosniff`: Prevenindo ataques de *MIME Sniffing*.

Proteções adicionais providas pelas camadas da Cloudflare e Vercel.

5.8 Rate Limiting e Disponibilidade

A avaliação da infraestrutura confirmou que a proteção contra Negação de Serviço (DDoS) e limitação de taxa (*Rate Limiting*) é gerenciada pelas camadas de borda (*Edge*) da Vercel e Cloudflare. Os testes não identificaram *endpoints* críticos expostos que pudessem ser explorados para exaustão de recursos do servidor.

5.9 Mitigação de Roubo de Credenciais

O uso do 2FA bloqueou tentativas de acesso mesmo quando credenciais (login/senha) válidas foram utilizadas em um ambiente de ataque simulado

6. CONCLUSÃO

O presente trabalho atingiu seu objetivo geral ao desenvolver e validar a plataforma "Gestão Atípicos", uma solução tecnológica voltada para a otimização do acompanhamento escolar de alunos com desenvolvimento atípico. A pesquisa demonstrou que a aplicação de metodologias ágeis, aliada a uma arquitetura *serverless* moderna, é capaz de entregar software de alta disponibilidade e baixo custo de manutenção para o contexto educacional. Do ponto de vista técnico, a escolha da stack tecnológica — composta por React, Vercel e Supabase — mostrou-se assertiva. A implementação de políticas de segurança em nível de banco de dados (*Row Level Security* - RLS) garantiu que a privacidade dos dados sensíveis fosse preservada nativamente, atendendo aos princípios fundamentais da Lei Geral de Proteção de Dados (LGPD). Os ciclos de testes de segurança, detalhados neste estudo, validaram a robustez da aplicação. A ausência de vulnerabilidades críticas (como *SQL Injection* ou *XSS*) e a eficácia dos mecanismos de autenticação e autorização evidenciam que a estratégia de *Security by Design* foi aplicada com sucesso. O sistema não apenas organiza informações, mas assegura a integridade digital de um público vulnerável.

Além da contribuição técnica, o projeto apresenta relevância social ao propor a substituição de processos manuais e fragmentados por uma gestão centralizada e eficiente. A ferramenta facilita a comunicação entre cuidadores e gestores, permitindo um acompanhamento mais próximo e humanizado da evolução dos alunos.

6.1 Limitações e Trabalhos Futuros:

Para a continuidade da pesquisa e evolução do software, sugerem-se as seguintes implementações futuras:

Integração IoT e Wearables: Sincronizar dados biométricos (como batimentos cardíacos e variabilidade da frequência cardíaca) captados por relógios inteligentes (smartwatches) diretamente com o Diário de Saúde, visando antecipar e prevenir crises sensoriais.

Notificações Multicanal: Expansão do módulo de comunicação para incluir o disparo automatizado de alertas críticos (como emergências médicas ou atrasos no check-in) via integrações com APIs de SMS e WhatsApp.

Expansão da Auditoria: Implementação completa de um sistema SIEM (Security Information and Event Management) dedicado, para monitoramento contínuo e alerta proativo de ameaças ao ambiente de produção.

REFERÊNCIAS

BRASIL. [Portal da Legislação]. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, [2015]. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/113146.htm. Acesso em: 15 dez. 2026.

BRASIL. [Portal da Legislação]. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, [2018]. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2017-2020/2018/lei/113709.htm. Acesso em: 15 dez. 2026.

BRASIL. **Decreto nº 10.502, de 30 de setembro de 2020**. Institui a Política Nacional de Educação Especial: Equitativa, Inclusiva e com Aprendizado ao Longo da Vida. Brasília, DF: Presidência da República, 2020.

LIMA, Marcos Guilherme Oliveira. **Gestão Atípicos: Sistema Web para Gestão Especializada de Alunos Atípicos na Educação Inclusiva**. 2026. Repositório de código (GitHub). Disponível em: https://github.com/z12guilherme/gestao_atipicos. Acesso em: 19 dez. 2026.

MICROSOFT. **The STRIDE Threat Model**. Azure Security Center, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>. Acesso em: 15 dez. 2026.

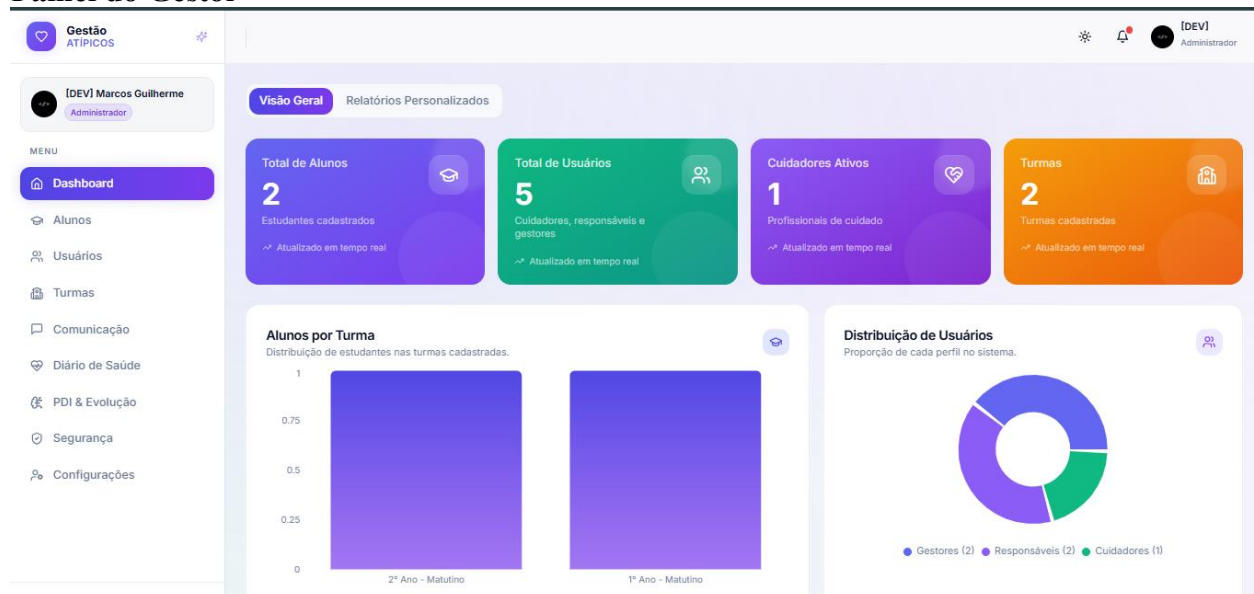
OWASP FOUNDATION. **Application Security Verification Standard (ASVS)**. Version 4.0.3. 2021. Disponível em: <https://owasp.org/www-project-application-security-verification-standard/>. Acesso em: 15 dez. 2026.

OWASP FOUNDATION. **OWASP Top 10:2021**. The Most Critical Web Application Security Risks. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 15 dez. 2026.

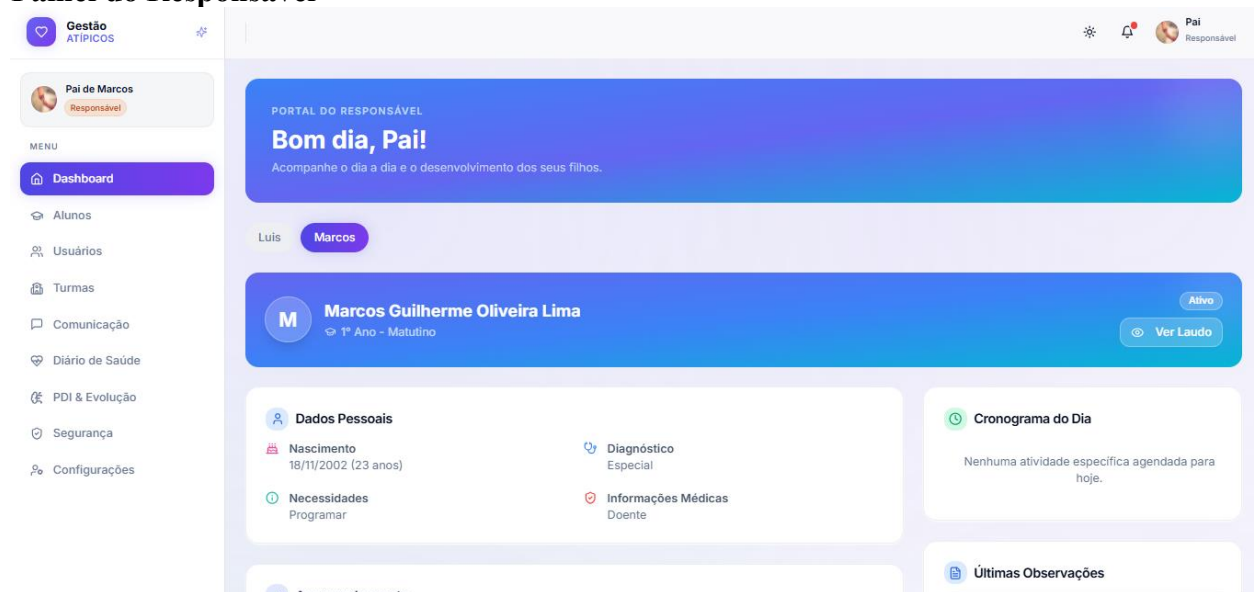
SUPABASE INC. **Row Level Security**. Documentation. 2026. Disponível em: <https://supabase.com/docs/guides/auth/row-level-security>. Acesso em: 15 dez. 2026.

Anexo A – Capturas de Tela

Painel do Gestor



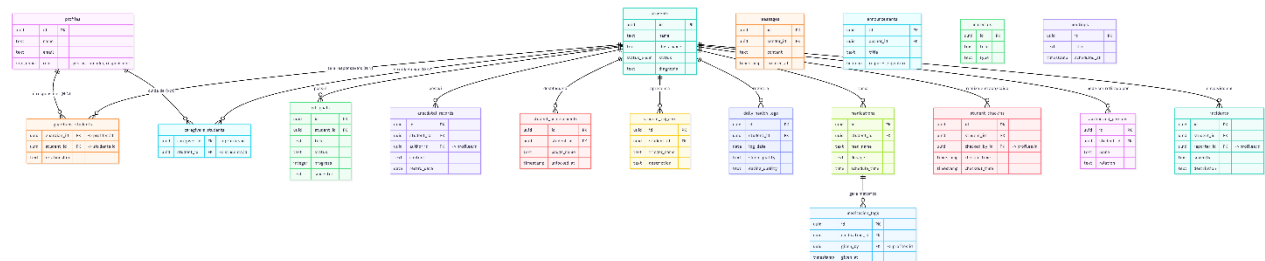
Painel do Responsável



Painel do Cuidador



Diagrama do Banco de Dados



Anexo B – Relatório de Pentest

1. DESCRIÇÃO DO PROJETO

O sistema Gestão Atípicos consiste em uma plataforma web hospedada no Vercel, utilizando o Supabase como backend para autenticação, API REST e banco de dados.

Status de Segurança: Auditado e Aprovado (Dezembro/2026).

2. COMPROMISSO COM A SEGURANÇA

A segurança da informação constitui prioridade no desenvolvimento da aplicação Gestão Atípicos. O presente anexo descreve os testes de segurança realizados, os controles implementados, as vulnerabilidades não identificadas e o processo de reporte responsável.

3. DOCUMENTAÇÃO DE SEGURANÇA RELACIONADA

Para consulta de detalhes técnicos adicionais, encontram-se disponíveis no repositório do projeto os seguintes documentos complementares:

- Modelo de Ameaças STRIDE (arquivo THREAT_MODEL_STRIDE.md)
- Checklist OWASP Application Security Verification Standard – ASVS (arquivo OWASP_ASVS_CHECKLIST.md)
- Playbook de Resposta a Incidentes com Honeytokens (arquivo HONEYTOKEN_PLAYBOOK.md)
- Política de Segurança para Escolas (arquivo POLITICA_SEGURANCA_ESCOLA.md)
- Conformidade com a Lei Geral de Proteção de Dados Pessoais – LGPD (arquivo LGPD_COMPLIANCE.md)

4. ESCOPO DOS TESTES DE PENETRAÇÃO

Os testes foram realizados exclusivamente no ambiente do projeto, sem qualquer impacto a sistemas ou dados de terceiros.

4.1 Componentes Avaliados

- Frontend hospedado no Vercel.
- API pública.
- Autenticação via Supabase Auth com Autenticação de Duas Etapas (2FA/TOTP).

- API REST do Supabase.
- Cabeçalhos HTTP (headers).
- Controle de acesso (Row Level Security - RLS).
- Validação de entradas.
- Proteções contra os principais riscos do OWASP Top 10.

5. TESTES REALIZADOS

5.1 Autenticação e Autorização

Foram realizadas tentativas de login com usuários inexistentes, senhas incorretas, e-mails válidos e inválidos, além de testes de enumeração de usuários, recuperação de senha e utilização direta da API de autenticação.

Resultados obtidos:

- Não foi detectada enumeração de usuários.
- Foram implementadas mensagens de erro genéricas.
- A autenticação encontra-se protegida por chave pública válida (anon key).
- Tokens inválidos foram rejeitados corretamente.
- O uso do 2FA bloqueou tentativas de acesso mesmo quando credenciais (login/senha) válidas foram utilizadas em um ambiente de ataque simulado.

5.2 Exposição de Chaves Supabase

Identificou-se a presença da anon key no código frontend.

Análise realizada: A anon key é pública por design da arquitetura Supabase e não concede acesso privilegiado, dependendo exclusivamente das políticas de Row Level Security (RLS).

Resultados obtidos:

- Não foi obtido qualquer acesso não autorizado.
- As políticas RLS impediram leitura e escrita indevida.
- As tentativas resultaram em resposta 401 Unauthorized.

Conclusão: Não configura vulnerabilidade.

5.3 JWT e Tokens

Foram testados JWT incompleto, reaproveitado, sem assinatura válida e cabeçalhos Authorization forjados.

Resultados obtidos:

- Tokens inválidos foram rejeitados.
- Não foi possível qualquer escalada de privilégio.

5.4 API REST Supabase

Realizaram-se tentativas de acesso direto ao endpoint /rest/v1/ utilizando apikey inválida, JWT inválido ou sem autenticação.

Resultados obtidos:

- Acesso negado (401 Unauthorized).
- Nenhuma tabela foi exposta publicamente.

5.5 Escalação de Privilégios

Tentou-se escalação vertical (exemplo: de cuidador para gestor), manipulação do parâmetro role via requisição PATCH /profiles e Mass Assignment em campos sensíveis.

Resultados obtidos:

- As políticas RLS bloquearam as alterações (0 linhas afetadas).
- O campo role foi ignorado silenciosamente pelo servidor.
- O perfil do usuário permaneceu inalterado.

5.6 Cross-Site Scripting (XSS)

Aplicaram-se payloads como <script>alert(1)</script>, <svg/onload=alert(1)> e variações em query string, rotas públicas e corpo JSON de requisições.

Resultados obtidos:

- Nenhum payload foi executado.
- Não foi observada reflexão de entrada.
- Rotas inexistentes retornaram código 404.

5.7 Cabeçalhos de Segurança

Cabeçalhos observados: Strict-Transport-Security, X-Content-Type-Options: nosniff, redirecionamento forçado para HTTPS, proteções Cloudflare e Vercel.

Resultados obtidos:

- Proteções modernas encontram-se ativas.
- Comunicação segura via TLS.

5.8 Rate Limiting e Proteção contra DDoS

A infraestrutura baseia-se em Vercel, Cloudflare e Supabase.

Resultados obtidos:

- Proteção gerenciada pela infraestrutura dos provedores.
- Nenhum endpoint crítico exposto.

5.9. VULNERABILIDADES NÃO IDENTIFICADAS

- Injeção SQL
- XSS (refletido ou armazenado)
- CSRF
- Quebra de autenticação
- Enumeração de usuários
- Exposição de dados sensíveis
- Escalada de privilégios
- Acesso indevido via API
- Vazamento de tokens válidos

5.91 DEFESA ATIVA E MONITORAMENTO

O projeto implementa estratégia de defesa em profundidade, incluindo:

- Honeytokens: credenciais falsas inseridas intencionalmente para detecção de vazamentos ou varreduras.
- Canary endpoints: rotas monitoradas que simulam endpoints sensíveis.
- SIEM dedicado: infraestrutura de monitoramento em tempo real (repositório: https://github.com/z12guilherme/gestao_atipicos-siem).
- Correlation ID: identificador único para rastreabilidade em logs.

5.92. ROADMAP DE TESTES FUTUROS

Planejam-se para ciclos futuros:

1. Testes de upload e storage (arquivos maliciosos, Stored XSS via SVG, DoS via imagens de grandes dimensões).
2. Verificação de race conditions em operações críticas.

- 3. Auditoria periódica de dependências (npm audit).

5.93. CONCLUSÃO

Os testes realizados demonstram que a aplicação Gestão Atípicos apresenta elevado nível de maturidade em segurança, não tendo sido identificadas vulnerabilidades exploráveis. A arquitetura adotada (Vercel + Supabase com RLS) alinha-se às boas práticas modernas de segurança da informação.

5.94. PROCESSO DE REPORTE RESPONSÁVEL

Caso seja identificada qualquer falha de segurança, recomenda-se abrir issue privada no repositório ou contatar diretamente o mantenedor do projeto. Não deve ser realizada exploração ou divulgação pública sem coordenação prévia.

5.95. HISTÓRICO DE AUDITORIAS

Data	Tipo de Teste	Escopo	Resultado
15/12/2026	Pentest Blackbox	Full Stack (Auth, RLS, API)	0 Vulnerabilidades Críticas

Última revisão: 08 de Agosto de 2026 **Status:** Aplicação aprovada nos testes realizados.